Shiksha Mandal's
Bajaj Institute of Technology, Wardha
(Recognized under section 2(f) of UGC Act 1956)
(Affiliated to DBATU, Lonere, Raigad, Maharashtra)

Academic Session: 2026-27
BITACAD/OS/COMP/2026-27/Training/VII/3
Date: 02/09/2026

Department of Computer Engineering

Technical Interview Question & Model Answer Bank

Academic Year	**2026-2027**	**Class**	**4th year**
Subject/Expert Area	**Operating Systems (Last updated on: 23/09/2026)**		
Expert Name	**Dr. A. N. Kazi**		

Sr. No.	Question	Model Answer	Practical Example
1	What is an operating system and what does it do?	An Operating System (OS) is the software layer between application programs and computer hardware. It manages the CPU, memory, storage, devices, files, and processes, and provides services to applications through system calls. Its major responsibilities include process management, memory management, file-system management, I/O and device management, resource allocation, and protection.	Opening Google Chrome creates a process managed by the OS.
2	What is the difference between kernel mode and user mode?	Kernel mode is a privileged CPU mode in which the operating-system kernel can execute privileged instructions and access hardware and protected memory. User mode is restricted and is where normal application programs execute. When an application needs a privileged operation, it requests the kernel through a system call.	Saving a file uses kernel services through system calls.
3	What is a system call and can you name a few?	A system call is the mechanism used by a user program to request a service from the operating-system kernel. Examples include open(), read(), write(), close(), fork(), exec(), wait(), and mmap(). For example, opening or reading a file ultimately requires kernel services.	A text editor and browser running separately are processes; tabs may use threads.
4	What is the difference between a process and a thread?	A process is a program in execution with its own address space and resources. A thread is a unit of execution within a process. Threads belonging to the same process share memory and many resources, making communication efficient, but they also require synchronization. Processes provide stronger isolation but are generally more expensive to create and communicate	A music player waiting for a song file to load enters a waiting state.

		between.	
5	What are the states a process goes through?	A process commonly moves through New, Ready, Running, Waiting/Blocked, and Terminated states. New means it is being created; Ready means it is waiting for CPU time; Running means it is executing; Waiting/Blocked means it is waiting for I/O or an event; Terminated means execution has completed. A process that finishes I/O normally returns to Ready before being scheduled again.	Switching CPU execution from Chrome to VS Code is a context switch.
6	What is a context switch and why does it cost time?	A context switch occurs when the CPU stops executing one process or thread and starts another. The OS saves the current execution context, such as registers, program counter, and stack pointer, and loads the context of the next task. Context switches create overhead because no application work is performed during the switch, and cache/TLB effects can further reduce performance.	Multiple apps compete for CPU time, so the scheduler decides who runs next.
7	What is CPU scheduling and why is it needed?	CPU scheduling is the process by which the OS selects a ready process or thread to execute next and determines how CPU time is shared. It is needed because many tasks compete for a limited number of CPUs. A good scheduler balances throughput, response time, fairness, CPU utilization, and avoidance of starvation.	Round Robin scheduling is used in many time-sharing systems.
8	Compare common CPU scheduling algorithms.	FCFS (First-Come, First-Served) executes jobs in arrival order and is simple, but can suffer from the convoy effect. SJF (Shortest Job First) selects the shortest job and can minimize average waiting time, but may starve long jobs. Round Robin assigns each process a time quantum and provides fairness and responsiveness, although very small quanta cause more context-switch overhead. Priority scheduling runs higher-priority processes first but can starve low-priority processes unless aging is used.	Windows can interrupt a running task to run a higher-priority task.
9	What is the difference between preemptive and non-preemptive scheduling?	In preemptive scheduling, the OS can interrupt a running process and give the CPU to another process, for example when a time quantum expires or a higher-priority process becomes ready. In non-preemptive	Running a large application with limited RAM uses virtual memory.

		scheduling, a process normally keeps the CPU until it blocks, voluntarily yields, or finishes. Round Robin is preemptive; FCFS and basic SJF are non-preemptive.	
10	What is virtual memory?	Virtual memory gives each process a private, large, contiguous-looking address space that is mapped to physical memory by the OS and hardware. Pages that are not currently needed can be kept on secondary storage. Virtual memory provides process isolation, allows programs to use an address space larger than available RAM, and simplifies memory management.	Memory pages of a program can be stored in non-contiguous RAM frames.
11	What is paging?	Paging divides virtual memory into fixed-size pages and physical memory into fixed-size frames. A page table maps virtual pages to physical frames, allowing a process's memory to be stored non-contiguously. Paging removes external fragmentation, but can introduce internal fragmentation and requires address translation; the TLB helps make this translation faster.	A 10 KB file stored in a 12 KB block wastes 2 KB internally.
12	What is the difference between internal and external fragmentation?	Internal fragmentation is unused space inside an allocated block. For example, if a fixed-size block is larger than the requested data, the unused portion is internal fragmentation. External fragmentation occurs when free memory exists but is split into scattered regions that cannot satisfy a large contiguous request. Paging avoids external fragmentation but can have internal fragmentation.	NTFS organizes files and folders on Windows.
13	What is a file system and what does it manage?	A file system is the structure and set of rules used by an OS to organize and store data on persistent storage. It manages files, directories, metadata such as permissions, timestamps and size, block allocation, free-space tracking, and access control. Examples include ext4 and XFS on Linux, NTFS on Windows, and APFS on macOS.	/home/user/file.txt is absolute; docs/file.txt is relative.
14	What is the difference between an absolute and a relative path?	An absolute path identifies a file from the root of the file system and does not depend on the current working directory. Examples are /home/user/data.txt on Linux and C:\Users\data.txt on Windows. A relative path is interpreted from the current	A web server handling many requests demonstrates concurrency.

		working directory, such as docs/report.txt or ../config.	
15	What is the difference between concurrency and parallelism?	Concurrency means managing progress on multiple tasks by interleaving their execution; it can occur even on a single CPU. Parallelism means executing multiple tasks at the same time, normally using multiple CPU cores or processors. In short, concurrency is about dealing with multiple tasks, while parallelism is about simultaneous execution.	Two processes waiting forever for each other's locked resources cause deadlock.
16	What is a deadlock and what conditions cause it?	Deadlock is a state in which a group of processes or threads wait indefinitely for resources held by one another. The four necessary Coffman conditions are mutual exclusion, hold and wait, no preemption, and circular wait. If any one of these conditions is prevented, deadlock cannot occur.	Multitasking lets you browse while listening to music.
17	What is the difference between multiprogramming, multitasking, and multiprocessing?	Multiprogramming keeps multiple programs in memory so the CPU can switch to another program when one is waiting for I/O. Multitasking extends this idea using rapid time slicing so multiple programs appear to run simultaneously. Multiprocessing uses multiple CPUs or cores so tasks can genuinely execute in parallel.	L1 cache stores recently used instructions for fast access.
18	Why does the CPU have caches, and how do they help?	CPU caches are small, fast memories located close to the processor. They store frequently or recently used instructions and data so the CPU does not always have to wait for slower RAM. Caches work well because programs exhibit temporal locality (reuse of recent data) and spatial locality (access to nearby data).	A video stream buffer smooths playback; printer queues use spooling.
19	What are buffering and spooling?	Buffering temporarily stores data while it moves between components with different speeds, such as a producer and consumer. Spooling queues jobs for a slow or shared device, traditionally a printer, usually on disk so multiple applications can submit jobs without waiting for the device. Buffering smooths a transfer; spooling manages a queue of jobs.	Two threads updating the same bank balance can create a race condition.
20	What is a race condition	A race condition occurs when the result of	A parking lot with 10 spaces

	and how do you prevent it?	concurrent execution depends on the timing of multiple threads or processes accessing shared data. For example, two threads can read the same counter value and both update it, causing an increment to be lost. It can be prevented by protecting the critical section with synchronization mechanisms such as mutexes, locks, or atomic operations.	is like a counting semaphore.
21	What is the difference between a mutex and a semaphore?	A mutex is primarily a mutual-exclusion lock. It has ownership: the thread that locks it is normally responsible for unlocking it. A semaphore is a counter used for signaling or controlling access to a limited number of resources. A counting semaphore can allow up to N concurrent users, while a binary semaphore has two states but does not have the same ownership semantics as a mutex.	A producer adds tasks to a queue while consumers process them.
22	Explain the producer-consumer problem and how you'd solve it.	The producer-consumer problem involves producers placing items into a shared bounded buffer and consumers removing them. Producers must wait when the buffer is full, consumers must wait when it is empty, and access to the buffer must be synchronized. A standard solution uses a mutex to protect the buffer plus condition variables or semaphores representing available space and available items.	Databases may detect and recover from deadlocks.
23	How does an OS handle deadlocks?	An OS can handle deadlocks through prevention, avoidance, detection and recovery, or by ignoring them. Prevention deliberately breaks one of the necessary deadlock conditions. Avoidance grants resources only when the resulting state remains safe, as in the Banker's algorithm. Detection allows deadlocks and searches for them, followed by recovery such as terminating or rolling back processes. Some general-purpose systems choose to ignore certain deadlocks when prevention would be too costly.	A bank checking safe resource allocation illustrates Banker's algorithm.
24	What is the Banker's algorithm?	Banker's algorithm is a deadlock-avoidance algorithm. Before granting a resource request, it checks whether the resulting allocation leaves the system in a safe state, meaning there is an order in which all processes can still complete. It requires	LRU keeps recently used pages and removes older unused ones.

		information about each process's maximum resource requirements. It is mainly a teaching and theoretical model because such maximum requirements are often difficult to know in practice.	
25	What are page replacement algorithms and why do they matter?	When physical memory is full and a required page is not resident, the OS must choose a page to remove. The page replacement algorithm determines which page is evicted and therefore influences the number of page faults. Optimal replacement chooses the page whose next use is farthest in the future and is mainly a benchmark. FIFO removes the oldest page and can suffer Belady's anomaly. LRU removes the least recently used page, while Clock/Second-Chance approximates LRU with lower tracking cost.	Too many memory-hungry applications can cause thrashing.
26	What is thrashing and how do you fix it?	Thrashing occurs when the system spends most of its time handling page faults and moving pages between RAM and disk rather than doing useful work. It commonly happens when the working sets of active processes cannot fit in physical memory. Solutions include reducing the degree of multiprogramming, using working-set-based allocation, and adding more RAM.	The TLB speeds up address translation during program execution.
27	What is the TLB and why is it important?	The Translation Lookaside Buffer (TLB) is a small, fast cache that stores recently used virtual-to-physical address translations. A TLB hit avoids a page-table lookup in memory, reducing the cost of virtual-address translation. A TLB miss requires a page-table lookup and then the translation can be cached.	A page is loaded only when first accessed.
28	What is demand paging?	Demand paging loads a page into physical memory only when the process actually accesses it. The first access to a non-resident page causes a page fault, after which the OS obtains the page from secondary storage and places it in a frame. It reduces unnecessary memory use and can improve startup time, but page faults are expensive.	Code, data, and stack segments are examples of segmentation.
29	What is the difference between segmentation	Paging divides memory into fixed-size pages and frames and uses mechanical	Chat applications use sockets for IPC.

	and paging?	address translation. Segmentation divides a program into variable-sized logical units such as code, data, and stack segments. Paging avoids external fragmentation, whereas segmentation can suffer from external fragmentation. Segmentation also naturally represents logical program units and can support protection by segment.	
30	What are the main inter-process communication mechanisms?	Common IPC mechanisms include pipes/FIFOs, message queues, shared memory, and sockets. Pipes provide byte streams, message queues provide structured kernel-managed messages, shared memory allows processes to access a common memory region and can be very fast, and sockets support local or network communication. Shared memory generally requires explicit synchronization to avoid races.	Kernel threads allow true multicore execution.
31	What is the difference between user-level and kernel-level threads?	User-level threads are managed by a user-space library and may be faster to create and switch because the kernel does not manage each thread directly. However, a blocking system call can block the whole process in some user-level models, and such threads cannot necessarily run in parallel across cores. Kernel-level threads are known and scheduled by the OS, can run in parallel on multiple cores, and can block independently, but have greater management overhead.	A low-priority task holding a lock delays a high-priority task.
32	What is priority inversion and how is it solved?	Priority inversion occurs when a high-priority task waits for a resource held by a low-priority task, while a medium-priority task prevents the low-priority task from running and releasing the resource. A common solution is priority inheritance: the low-priority task temporarily inherits the waiting high-priority task's priority until it releases the resource.	Spinlocks are common in kernel code for very short waits.
33	What is the difference between a spinlock and a mutex?	A spinlock makes a waiting thread repeatedly check whether a lock has become available, consuming CPU time but avoiding the sleep/wakeup overhead. A mutex can put a waiting thread to sleep until the lock becomes available. Spinlocks are useful when the expected wait is	A terminated child process awaiting wait() becomes a zombie.

		extremely short on multiprocessor systems; mutexes are preferable when waiting may be longer or when spinning would waste CPU resources.	
34	What are zombie and orphan processes?	A zombie process has completed execution but remains in the process table because its parent has not collected its exit status using wait(). It consumes a process-table entry but not normal CPU or memory resources. An orphan is a still-running child whose parent has terminated; it is adopted by a system process such as init/systemd.	DMA allows a disk controller to transfer data directly to RAM.
35	What is DMA and why does it matter?	Direct Memory Access (DMA) allows an I/O device to transfer data to or from main memory without requiring the CPU to handle every byte. The CPU configures the transfer, and a DMA controller performs the transfer and can interrupt the CPU when it is complete. This frees CPU time and improves high-throughput I/O performance.	Linux is largely monolithic; MINIX uses a microkernel design.
36	What is the difference between a monolithic kernel and a microkernel?	A monolithic kernel keeps major OS services such as memory management, device drivers, file systems, and networking in kernel space. This can provide good performance because components communicate through direct kernel calls, but a fault in a kernel component can affect the whole kernel. A microkernel keeps only essential mechanisms in the kernel and moves services such as drivers and file systems into user space. This improves fault isolation but can introduce IPC overhead.	fork() followed by modification of memory triggers copy-on-write.
37	What is copy-on-write and where is it used?	Copy-on-write (COW) allows multiple processes or components to share the same physical pages while they only read them. When one party attempts to modify a shared page, a protection fault occurs and the OS creates a private copy for the writer. A classic use is fork(), where the parent and child initially share pages rather than immediately duplicating all memory.	A shell uses fork() and exec() to launch commands.
38	How do fork and exec work together to run a new program?	fork() creates a new child process that initially resembles the parent and typically uses copy-on-write memory sharing. Both processes continue after fork(), with	Databases often use memory-mapped files for large datasets.

		different return values identifying parent and child. The child can then call exec() to replace its current program image with another executable. A shell commonly follows the fork → exec → wait pattern when launching a command.	
39	What are memory-mapped files and when are they useful?	A memory-mapped file uses mmap() to map file contents into a process's virtual address space. The program can then access the mapped region like memory, while the OS manages loading file-backed pages as needed. Memory mapping is useful for large random-access files, sharing data between processes, and loading executables or libraries. It can introduce page-fault costs and more complex error handling.	Two threads updating variables on the same cache line may suffer false sharing.
40	What are memory barriers and why are they needed?	A memory barrier or fence constrains the ordering of memory operations. CPUs and compilers can reorder operations for performance, but concurrent programs may depend on a particular visibility/order of reads and writes. Barriers help enforce the required ordering. Mutexes and atomic operations normally provide the necessary memory-ordering guarantees, while low-level lock-free programming may require explicit reasoning about memory ordering.	A keyboard press generates an interrupt.
41	What is the difference between interrupts and polling for I/O?	With polling, the CPU repeatedly checks a device to determine whether it needs service. This can waste CPU cycles but can provide predictable low latency. With interrupts, the device notifies the CPU when an event occurs, allowing the CPU to perform other work between events. Interrupts are efficient for infrequent events, while very high event rates can create interrupt overhead, so systems may use techniques such as interrupt coalescing or hybrid polling.	Keeping frequently used pages in RAM follows the working-set model.
42	What is the working set model?	The working set of a process is the set of pages it has actively used during a recent time window. If the working set is kept in physical memory, the process can execute with relatively few page faults. If the combined working sets of active processes exceed available RAM, the system may thrash. The model therefore helps control	Linux CFS tries to fairly distribute CPU time among tasks.

		the degree of multiprogramming and guide page replacement.	
43	How does a modern fair-share scheduler like Linux CFS work?	Linux's Completely Fair Scheduler (CFS) aims to distribute CPU time fairly among runnable tasks. It tracks a task's virtual runtime, which represents the amount of CPU time it has received with priority weighting, and generally chooses the runnable task with the smallest virtual runtime next. The runnable tasks are organized in a red-black tree ordered by virtual runtime, making selection efficient.	ext4 journaling helps recover after sudden power loss.
44	What is an inode and what does it store?	An inode is a Unix-like file-system data structure that represents a file and stores metadata such as size, permissions, owner, timestamps, link count, and references to the disk blocks containing the file's data. The file name is stored in a directory entry, which maps the name to an inode number. This separation explains concepts such as hard links and inexpensive directory-entry changes.	PC startup follows BIOS/UEFI → Bootloader → Kernel.
45	Walk me through what happens when a computer boots.	When a computer is powered on, firmware such as BIOS or UEFI initializes hardware and performs startup checks. It then transfers control to a bootloader, which loads the operating-system kernel and required initial data such as an initial RAM disk. The kernel initializes memory management, scheduling, device drivers, and the root file system, then starts the first user-space process such as init/systemd. That process starts services and the remaining user-space environment.	Many users reading a file while one user writes illustrates readers-writers.
46	What is the readers-writers problem?	The readers-writers problem is a synchronization problem involving a shared resource that can be read by multiple threads but must be exclusively accessed when being modified. Multiple readers can operate concurrently, while a writer must exclude both readers and other writers. Reader-writer locks are a common solution. A major design issue is fairness: reader preference can starve writers, while writer preference can starve readers.	NUMA-aware databases keep memory close to the CPU using it.
47	What is NUMA and why	NUMA (Non-Uniform Memory Access) is a	A web server thread pool

	does it matter for performance?	memory architecture common in multi-socket systems where each CPU or socket has local memory and remote memory belongs to another node. Accessing local memory is generally faster than accessing remote memory. Performance-sensitive systems therefore try to maintain CPU and memory affinity so threads frequently access memory close to the CPU running them.	processes incoming requests efficiently.
48	A system has three processes P1, P2, and P3. P1 arrives first but requires 10 ms CPU time, while P2 and P3 require only 2 ms each. If the system uses FCFS, what problem may occur?	P2 and P3 must wait for P1 to finish even though they require very little CPU time. This can increase average waiting time and response time. This situation is known as the convoy effect.	In a print server, one large printing job can delay many small printing jobs if they are handled strictly in arrival order.
49	A process with a CPU burst of 20 ms is running. A new process with a CPU burst of 2 ms arrives. Under SJF, should the new process immediately execute?	Standard SJF is non-preemptive, so the currently running process continues until completion. The new process executes afterward.	A batch-processing system may allow the current database operation to finish before executing a newly arrived short operation.
50	Suppose P1 requires 10 ms and P2 requires 2 ms. Both arrive at the same time. Which process should execute first under SJF and why?	P2 executes first because SJF selects the process with the shortest CPU burst. This generally minimizes average waiting time when burst times are known accurately.	Short file-processing jobs can be completed before large file-processing jobs.
51	A process has been waiting for a very long time because shorter processes continuously arrive. Which scheduling problem is this?	This is starvation. A process may wait indefinitely because other processes keep receiving CPU time.	In a priority-based system, a low-priority background process may never execute if high-priority processes continuously arrive.
52	A Round Robin system uses a time quantum of 2 ms. A process requires 10 ms of CPU time. How many CPU time slices will it receive?	It will receive 5 time slices of 2 ms each, assuming it is not blocked or interrupted for another reason.	Interactive applications receive short CPU intervals repeatedly so that the system remains responsive.
53	What happens if the Round Robin time quantum is extremely	Round Robin starts behaving similarly to FCFS because processes can complete their CPU bursts before the time quantum	If the quantum is 1 second while most processes require only 50 ms,

	large?	expires.	preemption rarely occurs.
54	A process performs CPU work for 5 ms, waits for I/O for 20 ms, and then requires another 5 ms of CPU time. Should the scheduler keep the process on the CPU during its I/O wait?	No. Once the process requests I/O, it becomes blocked or waiting, and the CPU can be assigned to another ready process.	While a process waits for disk data, the OS can execute another ready process.
55	A process has a burst time of 8 ms. Under Round Robin with a 3 ms quantum, explain its execution.	It receives CPU for 3 ms, is preempted, receives another 3 ms, and then receives the remaining 2 ms.	P1 executes for 3 ms, waits, executes for another 3 ms in a later round, and finally executes for 2 ms.
56	A scheduler always chooses the process with the shortest remaining CPU burst. Which algorithm is this?	This is Shortest Remaining Time First (SRTF), the preemptive version of SJF.	If P1 has 8 ms remaining and a new P2 requiring 3 ms arrives, P2 may preempt P1.
57	P1 is executing and has 6 ms remaining. A new P2 arrives with a CPU burst of 4 ms. What happens under SRTF?	P2 can preempt P1 because its remaining CPU burst (4 ms) is shorter than P1's remaining burst (6 ms).	A short interactive task can interrupt a longer computation under SRTF.
58	What information does an OS need to implement SJF effectively?	It needs an estimate of the next CPU burst time for each process. The exact future burst is generally not known, so operating systems may estimate it using previous behavior.	A process that historically performs short CPU bursts may be predicted to have another short burst.
59	A process arrives at time 5 ms and starts execution at 12 ms. What is its response time?	Response time = first CPU start time − arrival time = 12 − 5 = 7 ms.	A user launches an application at 5 ms and sees its first response at 12 ms.
60	A process arrives at 0 ms and completes at 20 ms. What is its turnaround time?	Turnaround time = completion time − arrival time = 20 − 0 = 20 ms.	A job submitted at 0 ms that finishes at 20 ms has a turnaround time of 20 ms.
61	A real-time application must complete a task before a strict deadline. Which scheduling concept becomes particularly important?	Deadline-aware real-time scheduling is important. Algorithms such as Earliest Deadline First (EDF) prioritize tasks according to their deadlines.	A control system may need to process sensor data before a specified deadline.
62	Two real-time tasks have deadlines of 20 ms and 50	The task with the earlier deadline, 20 ms, gets priority.	A control task due in 20 ms would be scheduled before a

	ms. Under EDF, which task gets priority?		task due in 50 ms.